Lab 11

CoAP and HTTP: Messages, Discovery and Retransmission

Communication Technologies for the Internet of Things | Lecture 11

Aim and context

Lecture 11 compares HTTP, CoAP and XMPP. This lab makes the most important differences between HTTP and CoAP observable: message size, resource discovery and reliability over UDP. For that, it uses a minimal CoAP server and client, written in Python, that exchange real messages on `localhost`.

The work needs no hardware and no virtual machine, and runs on any computer with Python 3. It is meant to be done during the class, at each student's pace, and is **not an assignment to hand in**.

Note

The program implements only what is needed: the base header, token, Uri-Path option and payload, as in RFC 7252. It does not replace a full library such as `aiocoap`.

Learning outcomes

- Interpret the octets of a CoAP message and identify its fields.
- Compare the size of a request and a response in CoAP and in HTTP.
- Discover the resources of a server through `.well-known/core`.
- Observe the retransmission of a confirmable message with exponential back-off.

Requirements

- A computer running Windows, macOS or Linux, with Python 3.9 or later.
- Estimated duration: 40 to 60 minutes.

How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. Read the whole step before running the first command. The boxes always have the same meaning:

Command to run

```
Command to run in the terminal.
```

Expected result (values may vary)

```
Result that should be displayed.
```

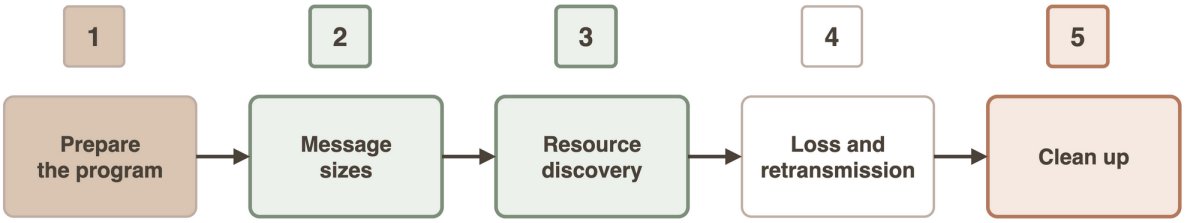
Analyse

A moment to pause and interpret the result, on your own or in discussion with colleagues.

Possible error and fix

An error that may occur and how to fix it.

Overview



A CoAP server and client written in Python, running on localhost

How to read the figure

The five steps read from left to right. The first prepares the program. Steps 2 and 3, in green, compare messages and discover resources. Step 4 observes retransmission with loss, and step 5 closes the work.

Step 1 — Preparing the program

Aim. Save the lab program.

Theoretical background

The program launches a CoAP server in a thread, on a UDP socket at `127.0.0.1`, and a client that builds raw messages, octet by octet. Each command, `sizes`, `discover` and `retransmit`, corresponds to a step of the guide. With no argument, it runs all three.

Create the file `coap_lab.py` with the contents below. The file is also in the `code` folder of the lecture material.

Contents of file `coap_lab.py`

```
"""CoAP vs HTTP lab (Lecture 11): message sizes, resource discovery and
    confirmable retransmission.

Run:  python3 coap_lab.py [sizes|discover|retransmit]
Only the Python standard library is used; everything runs on localhost.
"""
import socket
import struct
import sys
import threading
import time

# ---------------------------------------------------------------- CoAP codec (RFC
    7252, minimal)
CON, NON, ACK, RST = 0, 1, 2, 3
GET, CONTENT, NOT_FOUND = 1, 69, 132             # codes 0.01, 2.05, 4.04
RESOURCES = {"temp": b"23.5", ".well-known/core": b'</temp>;ct=0;rt="temperature
    "'}


def encode_options(path):
    """Uri-Path options (number 11), one per path segment."""
    out, prev = b"", 0
    for seg in path.split("/"):
        data = seg.encode()
        out += bytes([((11 - prev) << 4) | len(data)]) + data     # segments are
    short here
        prev = 11
    return out
```

```
def build(mtype, code, mid, token=b"", path="", payload=b""):
    head = struct.pack("!BBH", (1 << 6) | (mtype << 4) | len(token), code, mid)
    msg = head + token + (encode_options(path) if path else b"")
    return msg + (b"\xff" + payload if payload else b"")


def parse(data):
    ver_t_tkl, code, mid = struct.unpack("!BBH", data[:4])
    tkl = ver_t_tkl & 0x0F
    i, path, prev = 4 + tkl, [], 0
    while i < len(data) and data[i] != 0xFF:
        delta, ln = data[i] >> 4, data[i] & 0x0F
        prev += delta
        if prev == 11:
            path.append(data[i + 1:i + 1 + ln].decode())
        i += 1 + ln
    payload = data[i + 1:] if i < len(data) and data[i] == 0xFF else b""
    return dict(type=(ver_t_tkl >> 4) & 3, code=code, mid=mid, token=data[4:4 +
    tkl], path="/".join(path), payload=payload)


# ---------------------------------------------------------- server and client
def server(sock, drop_first=0):
    seen = 0
    while True:
        try:
            data, addr = sock.recvfrom(1500)
        except OSError:
            return
        seen += 1
        if seen <= drop_first:
            continue                                    # simulate a lost request
        req = parse(data)
        body = RESOURCES.get(req["path"])
        code = CONTENT if body is not None else NOT_FOUND
        sock.sendto(build(ACK, code, req["mid"], req["token"], payload=body or b
    ""), addr)


def start_server(drop_first=0):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(("127.0.0.1", 0))
    threading.Thread(target=server, args=(s, drop_first), daemon=True).start()
    return s, s.getsockname()


def get(addr, path, mid=1, timeout=0.5, tries=4):
    msg = build(CON, GET, mid, token=b"\x42", path=path)
```

```
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    for attempt in range(tries):
        c.settimeout(timeout * (2 ** attempt))        # exponential back-off
        c.sendto(msg, addr)
        t0 = time.time()
        try:
            data, _ = c.recvfrom(1500)
            return msg, data, attempt, time.time() - t0
        except socket.timeout:
            print(f"  attempt {attempt + 1}: no ACK within {timeout * 2 ** attempt
    :.1f} s, retransmitting")
    return msg, None, tries, None


# ------------------------------------------------------------ commands
def sizes():
    s, addr = start_server()
    msg, resp, _, _ = get(addr, "temp")
    http_req = (b"GET /temp HTTP/1.1\r\nHost: 192.168.1.50:8080\r\nUser-Agent:
    sensor/1.0\r\n"
                b"Accept: text/plain\r\nConnection: close\r\n\r\n")
    http_resp = (b"HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nContent-Length:
    4\r\n"
                 b"Server: sensor/1.0\r\nConnection: close\r\n\r\n23.5")
    print(f"CoAP request : {len(msg):3d} octets  {msg.hex(' ')}")
    print(f"CoAP response: {len(resp):3d} octets  {resp.hex(' ')}")
    print(f"HTTP request : {len(http_req):3d} octets (plus TCP: 3-way handshake
    and close)")
    print(f"HTTP response: {len(http_resp):3d} octets")
    p = parse(resp)
    print(f"parsed response: type={p['type']} code={p['code']} (2.05 Content)
    payload={p['payload']!r}")
    print(f"payload is {len(p['payload'])} octets of {len(resp)} in the CoAP
    response")
    s.close()


def discover():
    s, addr = start_server()
    for path in (".well-known/core", "temp", "humidity"):
        _, resp, _, _ = get(addr, path)
        p = parse(resp)
        cls = "2.05 Content" if p["code"] == CONTENT else "4.04 Not Found"
        print(f"GET /{path:<18} -> {cls}  {p['payload'].decode() or '-'}")
    s.close()


def retransmit():
```

```
    s, addr = start_server(drop_first=2)
    print("server drops the first two requests")
    msg, resp, attempt, dt = get(addr, "temp", timeout=0.5)
    print(f"answered after {attempt} retransmission(s); payload {parse(resp)['
    payload']!r}")
    s.close()


CMDS = {"sizes": sizes, "discover": discover, "retransmit": retransmit}

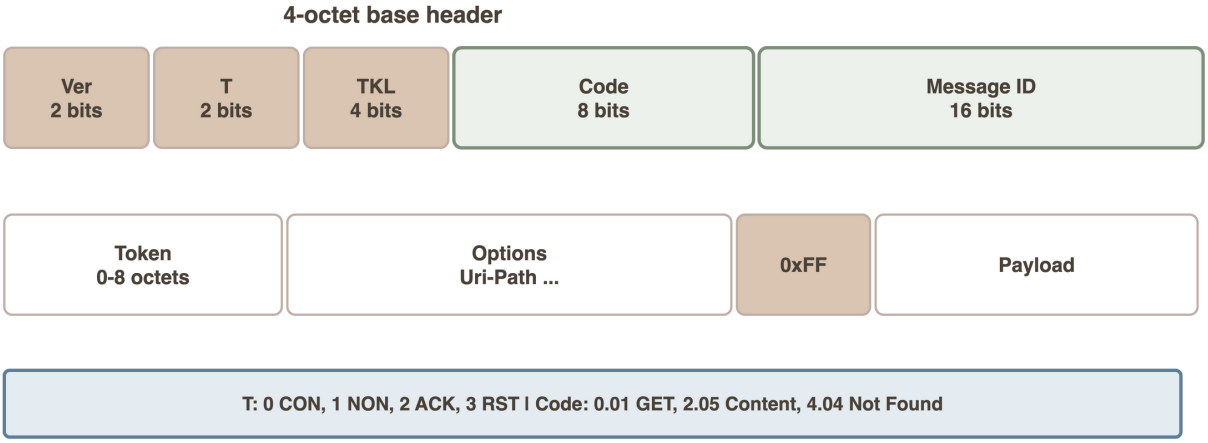
if __name__ == "__main__":
    which = sys.argv[1] if len(sys.argv) > 1 else None
    for name, fn in CMDS.items():
        if which in (None, name):
            print(f"== {name}")
            fn()
```

Possible error and fix

python3: command not found on Windows: use `python coap_lab.py sizes`.

Step 2 — Message sizes

Aim. Interpret the octets of a CoAP request and response and compare them with HTTP.



How to read the figure

The figure is the one from the slide: the 4-octet base header, followed by the token, options, the `0xFF` marker and the payload. Each field of the program output matches one of these boxes.

Theoretical background

The first octet `41` in hexadecimal means version 1, type 0 (confirmable) and token length 1. The second, `01`, is the GET code. The next two, `00 01`, are the message identifier. The token `42` follows, and the Uri-Path option, `b4 74 65 6d 70`, where `b` is delta 11 and `4` the length of the string `temp`. In the response, `61` indicates type ACK and `45` the code 2.05 (Content), followed by the marker `ff` and the payload `23.5`.

To analyse in this step: how many octets are header and how many are data.

Command to run

```
python3 coap_lab.py sizes
```

Expected result (values may vary)

```
CoAP request :  10 octets  41 01 00 01 42 b4 74 65 6d 70
CoAP response:  10 octets  61 45 00 01 42 ff 32 33 2e 35
HTTP request : 110 octets (plus TCP: 3-way handshake and close)
```

```
HTTP response: 107 octets
parsed response: type=2 code=69 (2.05 Content) payload=b'23.5'
payload is 4 octets of 10 in the CoAP response
```

Analyse

How many useful data octets does the CoAP response have, and what fraction of the total do they represent? The HTTP request shown is minimal. Which octets would a `Cookie` or a browser-like `User-Agent` add? And what did opening the TCP connection cost, in messages?

Step 3 — Resource discovery

Aim. Ask a server which resources it offers and observe a 4.04 error.

Theoretical background

CoAP defines a special resource, `.well-known/core`, which returns the list of the server's resources in the CoRE link format (RFC 6690). A client can thus discover what exists without prior knowledge. A non-existent resource returns code 4.04, equivalent to HTTP 404.

Command to run

```
python3 coap_lab.py discover
```

Expected result (values may vary)

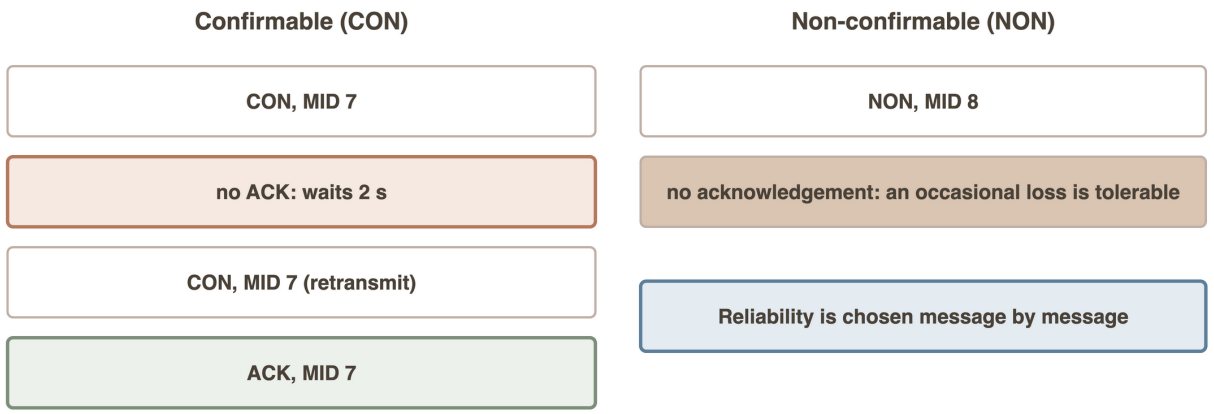
```
GET /.well-known/core   -> 2.05 Content  </temp>;ct=0;rt="temperature"
GET /temp               -> 2.05 Content  23.5
GET /humidity           -> 4.04 Not Found  -
```

Analyse

What does `ct=0` mean in the resource description? What advantage does discovery bring when installing dozens of sensors?

Step 4 — Loss and retransmission

Aim. Observe a confirmable message being retransmitted with exponential back-off.



How to read the figure

On the left, what the program does: it sends a confirmable message, receives no ACK, waits, retransmits and finally receives the confirmation. On the right, the non-confirmable alternative, which does not wait for a reply.

Theoretical background

In a confirmable message, the sender stores the message and starts a timer. If the ACK does not arrive, it retransmits with the same identifier and doubles the waiting time. The standard uses an initial value of about 2 seconds, with a random factor, and up to 4 retransmissions. The program uses 0.5 seconds, so the demonstration is fast. For the server, the same identifier makes it possible to recognise duplicates.

To analyse in this step: the waiting time at each attempt.

Command to run

```
python3 coap_lab.py retransmit
```

Expected result (values may vary)

```
server drops the first two requests
  attempt 1: no ACK within 0.5 s, retransmitting
  attempt 2: no ACK within 1.0 s, retransmitting
answered after 2 retransmission(s); payload b'23.5'
```

Analyse

Change `drop_first` to 5 and predict the result. When does the client give up? In which IoT applications would you use a non-confirmable message?

Step 5 — Cleaning up

Nothing was installed: just delete the file `coap_lab.py`.

Review questions

1. Which CoAP header fields let a response be matched to its request?
2. Why does CoAP not need to establish a connection?
3. How does a client discover a server's resources without knowing them?
4. What is the consequence of repeating the same message identifier in a retransmission?

Summary

- The CoAP **base header** takes 4 octets and is binary, unlike HTTP, which is textual.
- Without TCP, there is no **connection** and no three initial exchanges.
- **Discovery** is done at `.well-known/core`.
- **Reliability** is handled in the application: acknowledgement, retransmission and exponential back-off.